

MIE438 Project Report: Autonomous Cup Pong Robot

April 7, 2026

Abanoub Bashara (1010010612)

Solomon Pius (1009975127)

Roy Bou Abboud (1009905462)

Yiyi Xu (1009851402)

Demo link: <https://www.youtube.com/watch?v=Q0vxaehKVus>

Introduction & Project Evolution

The original project proposal outlined a five-phase plan to develop an autonomous cup pong robot capable of playing a perfect game without human intervention. Phases three to five focused on computer vision (CV) and user interface (UI) design. After receiving feedback on our proposal, we made the decision to eliminate the computer vision subsystem from the scope (see *docs/process/Instructor_Feedback.md*). The primary concern was integration complexity as well as course relevance. Developing a reliable vision pipeline and integrating it with a real-time embedded control stack introduced many points of failure and was far out of scope for this course's content.

Instead of CV, we decided to use a 1D LiDAR (TF-Luna), which provides direct distance measurement to the cups. By mounting the LiDAR on the rotating base, we are able to scan the entire playing field. This pivot allowed us to preserve the core sense-plan-act architecture of the robot while also allowing the team to focus on the embedded problems: sensor integration, motor power & control, and communication protocols.

The revised project scope targets completion of Phase 1 (MVP – repeatable ball launch at fixed distance) and Phase 2 (LiDAR-enabled autonomous aiming and launch at a fixed distance). The final turret robot boasts a servo-actuated base to scan the playing field and aim the turret, a LiDAR subsystem that finds the closest cup using minimum distance, dual brushless motors that spin flywheels to a set RPM, and a feeder motor to push the ball into the launcher.

Final System Architecture

The final system uses only a single ESP32 development board to consolidate all computation and real-time control (see *docs/electrical/ESP32_Pinout_Diagram.pdf*). This is a significant simplification from the original dual-platform architecture (Raspberry Pi + ESP32), which was proposed in the initial design. The elimination of CV also removed the need for a high-compute platform entirely. This also helped tighten integration between sensing and actuation since every signal is either generated or consumed directly by the ESP32. The final architecture follows a sense-plan-act paradigm (see *docs/software/Software_Architecture.png* and *docs/software/Firmware_Flowchart.png*).

For sensing, the TF-Luna sensor is mounted co-axially with the turret barrel to measure the cup distances and communicate with the ESP32 over I²C (see *firmware/src/main.cpp*, line 123). After powering on the system, the robot begins by sweeping across 5 to 120 degrees of the playing field (and back) in 2-degree increments and logging the smallest measured distance. Each step is delayed by 120ms to gather a reading, and the communication over I²C occurs at a very fast 400kHz frequency to ensure minimal latency. After an entire scan, the minimal distance to a cup and the angle at which it was found is passed as an input to other subsystems.

Next, the current version of our robot requires minimal planning. We had planned for a system that would use a full projectile model or interpolate distances to calculate the PWM (and hence speed) to reach a given cup, this could not be implemented due to faulty hardware. This would be a large improvement in future iterations if undertaken. Instead, the robot plans a servo rotation to the angle that recorded the shortest cup distance, and uses a fixed PWM for every shot (meaning shot distance was fixed as well) (see *firmware/src/main.cpp*, line 171-232).

For actuation, the ESP32 then commands the servo to slowly slew towards the selected angle at a max of 2 degrees per 25ms to avoid sudden movements (see *firmware/src/main.cpp*, line 137). This was needed as writing the angle to the servo directly would create a large torque, which could damage the structure or cause instability. After reaching the given angle, the ESP32 pushes PWM signals, which gradually increase into the flywheel motors via a L298N motor driver until the set value is reached. The flywheels first receive a PWM of 120/255 duty for 300ms to overcome the heavy load from the wheels and begin spinning. Then, the duty is dropped to 42/255 for 6 seconds to allow a proper spin-up, which is followed by a 150/255 duty to the feeder motor, which advances the ball into the launcher (see *firmware/src/main.cpp*, line 235-292).

Beyond that, the system is powered using a 12V LiPo battery for the motors, an LM2596 buck converter providing 5V to the servo & other peripherals, and the standard 3.3V output from the ESP32 for the remaining components.

Hardware Design

The 3D printed subsystem can be found in docs/mechanical/Final_CAD_Assembly.obj. This shows the final assembly of our 3D printed parts. From the final assembly document, the ball feeder tube was the only part not designed by us. It was taken from a GitHub repository created by Tylr-J42 <https://github.com/Tylr-J42/XRP-Ping-Pong-Ball-Shooter>. The used file can be found in Ping Pong Ball Shooter v29.step. All other parts of the CAD were created using Onshape.

Subsystem	Device	ESP32 pin(s)	Protocol
LiDAR	TF-Luna (0x10)	SDA=GPIO21, SCL=GPIO22	I2C @ 400 kHz
Turret	MG995 servo	GPIO4	50 Hz PWM (ESP32Servo)
Flywheel A	L298N EN_A/IN1/IN2	13 / 14 / 16	1 kHz PWM (LEDC ch 8)
Flywheel B	L298N EN_B/IN3/IN4	27 / 17 / 23	1 kHz PWM (LEDC ch 9)
Indexer	BTS7960 RPWM/LPWM/R_EN/L_EN	18 / 19 / 25 / 26	20 kHz PWM (LEDC ch 10)

Table 1: Hardware Subsystem and ESP32 Connections

The core compute system is the ESP32-Wroom-32E running Arduino Framework C++ compiled through PlatformIO. Power is supplied from a 12V battery pack with a separate regulated 5V for the servo and TF-Luna, and 12V for the motor drivers.

The flywheels share the L298N and run at the same duty; we were not able to complete an RPM sensor and variable control system (see Reflection / Conclusion section) to address this. The BTS7960 drives the indexer because it has significantly higher current headroom for the stall current during ball-push. Running the indexer to 20 kHz kept the motor out of the audible whine range, while the L298N flywheels run at 1 kHz, which the L298N's slower switching tolerates well.

A system schematic and full BOM are in docs/electrical/.

Firmware Design

All firmware can be found in *firmware/*. The PlatformIO project also defines one bring-up environment per peripheral: TF-Luna, IR beam, hall sensor, limit switches, flywheel, servo, and indexing motor. Each is compiling a single test file in *firmware/tests*. These programs were how we validated each subsystem before integration, and they remain the authoritative reference for each peripheral's wiring.

A firmware flowchart can be found in *docs/software/Firmware_Flowchart.png*

Use of ESP32 Peripherals

Course-relevant microcontroller features used:

- I²C (Wire) at 400 kHz, reading six bytes from TF-Luna register 0x00 per sample and extracting a little-endian 16-bit distance.
- LEDC (ledcSetup / ledcAttachPin / ledcWrite) for three independent hardware PWM channels at different frequencies, the only way to run the BTS7960 at 20 kHz and the L298N at 1 kHz simultaneously without software-PWM jitter.
- ESP32Servo (Arduino library on top of LEDC) for 50 Hz RC servo pulses at 500–2500 μ s.
- USB serial at 115 200 baud as both a human-readable log and a machine-readable CSV stream for offline polar plotting.

Libraries Used:

All developer-supplied libraries (Arduino core for ESP32, Wire, LEDC) are part of the Espressif Arduino framework, and per the course guidelines, do not require citation.

Third-party library: ESP32Servo - Kevin Harrington, John K. Bennett.

<https://github.com/madhephaestus/ESP32Servo>

Testing & Results

Each peripheral was validated independently using the test environments in firmware.

firmware/test/tfluna verified I²C bring-up and distance accuracy; *firmware/test/servo* swept the MG995 end-to-end; *firmware/test/radar_scan* validated the combined servo + LiDAR polar scan. The LiDAR polar scan was visualized by taking the LiDAR outputs, streaming them through serial to a Python program that visualized the output (see *firmware/tools/radar_view.py*). The servo then returns to the minimum distance measured by the LiDAR; *firmware/test/l298n* and *firmware/test/flywheel* validated the flywheel driver activation; and *firmware/test/all_motors* validated the full firing sequence end-to-end.

End-to-end performance metrics (see *docs/process/Performance_Metrics.pdf*)

- Cup detection rate: 100%
- Angular accuracy: +/- 2 degrees
- Maximum reliable shooting distance: 200cm

Course Content Connection

This project required many MIE438 concepts to complete on both the hardware and firmware sides. The ESP32 was selected due to its breadth of GPIO pins (see *docs/electrical/ESP32_Pinout_Diagram.pdf*), strong compatibility with hardware requiring both 3.3V and 5V, and its flexibility for integrating course concepts under one hood. All inputs, outputs, control signals and some powering were all coordinated through the ESP32.

In order for our robot to find the minimum distance from itself to a cup, we combined the I²C protocol with macros to create the cup detection subsystem. The I²C communication protocol was set up between the ESP32 and TF-Luna LiDAR, simplifying the transfer of data significantly and ensuring consistent readings for the robot to aim itself properly. A macro was used to combine two 8-bit (1 byte) signals read over I²C into a single 16-bit value to quickly construct the signal needed for the robot to choose the best shooting angle (see *firmware/src/main.cpp*, line 39). Similarly, inline functions were used for actions that were repeated multiple times and required quicker execution, such as setting the duty cycle for the flywheel or stopping all the motors at once (see *firmware/src/main.cpp*, line 107-118).

Additionally, the concept of pulse-width-modulation (PWM) was used in conjunction with motor drivers to control the three motors on our system: 1) the two flywheel motors and 2) the feeder motor. Varying PWM was initially used as a testing strategy to qualitatively determine the range of values that yield a feasible launch distance of the ball (see *docs/mechanical/Final_Assembly.png*). Then, PWM signals were generated from the ESP32 and delivered to the appropriate motors via both our dual-channel motor drivers. This concept was important to our system, as we were building a control loop that used RPM readings to adjust PWM in order to eliminate any variance between both motors (however, this system was never implemented due to faulty hardware).

Furthermore, we made use of a state machine to encode the robot's different stages during a shooting cycle. The three states were Scan, Aim, Fire, and Done. This greatly helped our system by breaking down phases within the sense-plan-act paradigm and allowing for easy debugging and readability. For example, timing the flywheel and feeder motors with the servo was much easier once we implemented a state machine that distinguishes between both phases (see *firmware/src/main.cpp*, line 235-292).

Finally, two additional PID controllers were planned for the system and would be the next improvements that incorporate course concepts. The first one would be a DIY RPM sensor using a magnet and hall effect sensor on the flywheel (see *firmware/tests/hall_sensor/main.cpp*). This system would take in RPM, use a precalculated "RPM vs Distance Travelled by Ball" chart to determine a PWM value for a required cup distance, and adjust accordingly using a tuned PID controller. Similarly, the built-in encoders of the feeder motor would be fed as an input to a second tuned PID, which would control the total rotation of the feeder, increasing consistency between shots.

Reflection & Conclusion

The two most consequential engineering design decisions we made for this project were the pivot away from computer vision and the consolidation to a single ESP32 platform. We believe both decisions were correct. Eliminating the CV pipeline allowed us to focus on the actual embedded

systems work, such as sensor integration, hardware assembly, design testing, and firmware architecture. The TF-Luna was a clean replacement that fit naturally with the ESP32 stack over I²C.

The most significant issues we encountered that limited our robot's capabilities were all related to mechanical or hardware rather than firmware. Firstly, the custom ball bearing we created for the rotating base caused turret tilting and, at times, instability during rotation (see *docs/mechanical/Final_Assembly.png*), creating a launch error that could never be reduced. Secondly, only 1 out of 2 hall sensors was proven to be effective, and there was no structurally sound way of attaching the neoprene magnets to the flywheel at high RPMs, hence abandoning the PID-controlled PWM for the motors was required. Thirdly, the infrared beam sensor we wanted to implement on the robot to determine the current state of the ball launch never worked during testing, and was therefore removed from the system (see *firmware/tests/ir_beam/main.cpp*). Finally, the PID-controlled feeder motor was also never implemented due to hardware issues related to its wiring, which unfortunately meant no physical control loops within our system. In the future, fixing these issues would be the biggest way of improving the robot's performance considerably. Beyond that, other upgrades that would strengthen our robot include using a 2D lidar as opposed to the servo spinning base, implementing a fully-fledged state machine for the entire autonomy stack, or successfully fitting an infrared beam to detect when the ball is launched.

In conclusion, we worked around our original ambitious goal and created a successful embedded system centred around a single ESP32 that correctly shot ping-pong balls into cups.

AI Usage Statement

In accordance with course guidelines, we disclose the use of AI tools during this project. Anthropic's Claude and Claude Code were used in two limited capacities: generating and refining inline code comments and documentation within the firmware source files, and assisting with debugging after firmware development, including diagnosing peripheral bring-up issues and interpreting unexpected sensor behaviour. All architectural decisions, hardware design, firmware logic, and testing were performed by the team. AI-generated suggestions were reviewed, validated, and modified by team members before being incorporated into the final codebase.